

LEVEL 7: Student Linked List

1. Objective

Maintain student records dynamically. This introduces nodes, pointers, malloc, linking and traversal.

2. Data Structure / Algorithm Used

Singly Linked List

3. Problem Statement

Extend the Library Management System by implementing the student linked list module. Complete this module independently before integrating it with previous levels.

4. Algorithm / Working

Allocate a node → enter student data → link it before head → update head → traverse next pointers.

5. C Program

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int id; char name[40]; struct Node *next;
} Node;

void insertFront(Node **head) {
    Node *p=malloc(sizeof(Node));
    if(p==NULL) { printf("Allocation failed\n"); return; }
    printf("Student ID: "); scanf("%d",&p->id);
    printf("Name: "); scanf(" %39[^\n]",p->name);
    p->next=*head; *head=p;
}

void display(Node *p) {
    while(p!=NULL) {
        printf("%d - %s\n",p->id,p->name);
        p=p->next;
    }
}

int main() {
    Node *head=NULL; int n,i;
    printf("Number of students: "); scanf("%d",&n);
    for(i=0;i<n;i++) insertFront(&head);
    printf("\nStudent List:\n"); display(head);
    return 0;
}
```

6. Sample Input

Number of students: 2
201 Rahul
202 Priya

7. Sample Output

202 - Priya
201 - Rahul

8. Complexity

Front insertion: $O(1)$. Traversal/search: $O(n)$. Dynamic storage: $O(n)$.

9. Practice Exercises

- Insert at end.
- Search by student ID.
- Delete a node.

10. Related Viva Questions

- What is a node?
- Why is next a pointer?
- Why use malloc?
- What is front insertion complexity?